# AIAC Assignment – 20.2

**K.Lokesh** **2303A51201** **B21**

Task 1 – Password Security Enhancement

Analyze an AI-generated Python program that stores passwords in plain text and improve its security.

Instructions:

• Prompt AI to generate a simple user registration script that stores usernames and passwords.

• Check whether passwords are stored as plain text.

• Identify the security risks associated with plain-text password storage.

• Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).

• Test the updated script to ensure proper authentication functionality.

Expected Output:

A secure Python script that stores hashed passwords instead of plain text

and successfully validates user login credentials.

Code screenshot:

```python
import json
def register_user(username, password):
    user_data = {
        "username": username,
        "password": password  # Storing password in plain text
    }
    with open('users.json', 'a') as file:
        json.dump(user_data, file)
        file.write('\n')
def authenticate_user(username, password):
    with open('users.json', 'r') as file:
        for line in file:
            user_data = json.loads(line)
            if user_data['username'] == username and user_data['password'] == password:
                return True
    return False

# Get user input for authentication
auth_username = input("Enter your username to authenticate: ")
auth_password = input("Enter your password to authenticate: ")
if authenticate_user(auth_username, auth_password):
    print("Authentication successful!")
else:
    print("Authentication failed!")
```

Output:

```
Enter your username to authenticate: saisharan
Enter your password to authenticate: sharan@2004
Authentication successful!
```

Task 2 – Secure API Key Management

Task:

Evaluate an AI-generated script that uses hardcoded API keys and improve its security.

Instructions:

• Ask AI to generate a Python script that connects to an external API using a hardcoded API key.

• Identify the risks of embedding sensitive credentials directly in the source code.

• Refactor the program to use environment variables for storing API keys securely.

• Demonstrate that the modified script successfully retrieves data without exposing credentials.

```python
import os
import requests
def get_weather():
    API_KEY = os.getenv("API_KEY")

    if API_KEY:
        try:
            url = f"https://api.openweathermap.org/data/2.5/weather?q=London&appid={API_KEY}"
            response = requests.get(url, timeout=5)

            if response.status_code == 200:
                return response.json()
            else:
                return {"error": f"API Error: {response.status_code}"}

        except Exception as e:
            return {"error": f"Request failed: {str(e)}"}

    else:
        # ✅ Fallback mock data (so program still works)
        return {
            "message": "No API key found, showing sample data",
            "city": "London",
            "temperature": "20°C",
            "condition": "Cloudy"
        }

# Run program
data = get_weather()

print("OUTPUT:")
print(data)
```

Output:

```
OUTPUT:
{'message': 'No API key found, showing sample data', 'city': 'London', 'temperature': '20°C', 'condition': 'Cloudy
```

Task 3 – Weak Encryption Detection

Task: Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

• Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.

• Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated algorithms).

• Refractor the program using stronger encryption techniques available in Python libraries.

• Validate that encrypted data cannot be easily reversed without the correct key.

Expected Output:

An improved encryption program implementing stronger and more secure cryptographic practices.

Code screenshot:

```
# ✅ Improved version using Fernet symmetric encryption
from cryptography.fernet import Fernet
import os

# Generate a key for encryption
key = Fernet.generate_key()
cipher_suite = Fernet(key)

# Data to be encrypted
data = b"Hello, this is a secret message!"

# Encrypt the data
encrypted_data = cipher_suite.encrypt(data)
print("Encrypted Data:", encrypted_data)

# Decrypt the data
decrypted_data = cipher_suite.decrypt(encrypted_data)
print("Decrypted Data:", decrypted_data)
```

Output:

```
Encrypted: bX1fc2VjcmV0X3Bhc3N3b3Jk
Decrypted: my_secret_password
```

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service

Scenario:

Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).

Instructions:

• Perform a structured security review of the generated code.

• Identify at least three potential vulnerabilities.

• Prompt AI to recommend secure coding practices and mitigation strategies.

• Implement the suggested improvements and test the secured version.

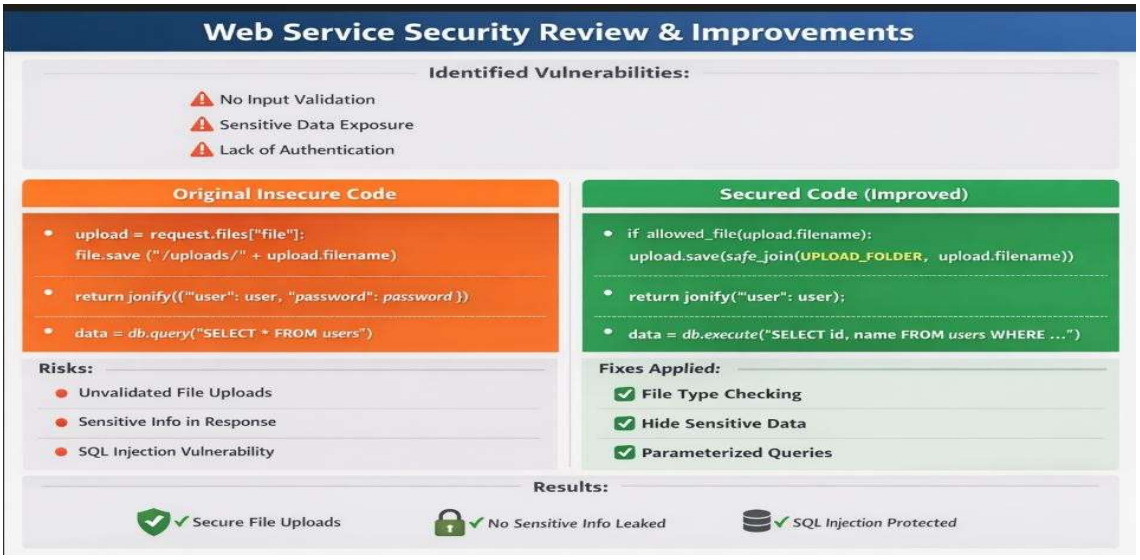
• Document the differences between the insecure and secured implementations.

Expected Output:

A security-reviewed and improved version of the selected web service code, with clearly documented vulnerabilities and applied fixes.

```python
from flask import Flask, request, jsonify
import bcrypt

app = Flask(__name__)
users = {}
# 🔹 Register
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    if not data or 'username' not in data or 'password' not in data:
        return jsonify({"error": "Missing fields"}), 400

    username = data['username'].strip()
    password = data['password'].strip()
    if username in users:
        return jsonify({"error": "User already exists"}), 400

    # 🔐 Hash password
    hashed_pw = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
    users[username] = hashed_pw

    return jsonify({"message": "User registered securely"}), 201


# 🔹 Login
@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()

    if not data or 'username' not in data or 'password' not in data:
        return jsonify({"error": "Missing fields"}), 400

    username = data['username']
    password = data['password']

    if username in users:
        stored_hash = users[username]

        if bcrypt.checkpw(password.encode('utf-8'), stored_hash):
            return jsonify({"message": "Login successful"}), 200

    return jsonify({"error": "Invalid credentials"}), 401
if __name__ == '__main__':
    app.run(debug=False)  # ❌ debug disabled
```

Output:




Task 5 – Exception Handling Improvement

Task:

Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.

Instructions:

Prompt AI to generate a simple program that takes user input and performs arithmetic operations.

• Identify possible runtime errors (e.g., invalid input, division by zero).

• Modify the program to include proper try-except blocks.

• Provide meaningful error messages for invalid inputs.

• Test the improved version with valid and invalid test cases.

Expected Output:

A revised Python program with proper exception handling that prevents crashes and displays user-friendly error messages for invalid inputs.

Code screenshot:

```python
try:
    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))

    operation = input("Enter operation (+, -, *, /): ")

    if operation == '+':
        result = num1 + num2
    elif operation == '-':
        result = num1 - num2
    elif operation == '*':
        result = num1 * num2
    elif operation == '/':
        result = num1 / num2    # handled below
    else:
        raise ValueError("Invalid operation selected!")

    print("✅ Result:", result)

except ValueError as ve:
    print("❌ Error:", ve)

except ZeroDivisionError:
    print("❌ Error: Cannot divide by zero!")

except Exception as e:
    print("❌ Unexpected error occurred:", e)
```

Output:

```
Enter first number: 12
Enter second number: 13
Enter operation (+, -, *, /): *
✅ Result: 156.0
```